Tecnologie Web

Informatica per il Management

1. World Wide Web

Il World Wide Web si basa su un'architettura client - server:
- Il client chiede le risorse al server (non deve avere per forza una interfaccia grafica)
- Il server è in attesa di richieste e, ricevuta una richiesta di risorsa, la restituisce al client
- Esistono altre entità intermedie che possono intervenire in questo flusso di comunicazione, ad esempio il proxy

Il browser è un client Web in grado di visualizzare i documenti e interagire con l'utente durante la navigazione.

Alla base di WWW ci sono tre protocolli / linguaggi:
- URI: standard per identificare in maniera generale risorse di rete e per poterle specificare all'interno di documenti ipertestuali.
- HTTP: protocollo di comunicazione stateless e client / server per l'accesso a risorse ipertestuali.
- HTML: linguaggio per la marcatura di documenti ipertestuali basata su SGML e XML.

A questi si aggiungono molti altri standard, diventati sempre più centrali nello sviluppo di applicazioni Web: CSS, JavaScript o TypeScript e JSON.

1.1. La nascita del WWW

Il WWW nasce nel 1989 da un'idea di Tim Berners-Lee, Robert Cailliau e altri ricercatori del CERN, il centro di ricerca di fisica nucleare di Ginevra.
L'obiettivo era quello di costruire una piattaforma per la diffusione e ricerca di articoli e informazioni tra ricercatori, per realizzarla identificarono tre pilastri: Internet, Ipertesti e SGML (un metalinguaggio per descrivere formati di interscambio di dati e documenti).
Negli anni successivi il WWW si afferma su scala mondiale grazie al browser Mosaic e Netscape, successivamente Microsoft introduce alcuni miglioramenti su HTML per aumentare l'appeal del proprio browser, si affermano così i Browser di Microsoft (Explorer) e da Netscape nascerà poi Firefox.
Per evitare guerre commerciali i fondatori fondarono il 3WC, un consorzio che prevedeva la partecipazione di aziende, enti di ricerca, università, istituzioni pubbliche e private. Questo ente si è sempre battuto per garantire accessibilità al Web da parte di tutti e libertà d'espressione.
Successivamente questo ente ha diretto lo sviluppo dei più importanti standard come HTML, CSS, HTTP, XML, WAI etc.
Il W3C forma gruppi di lavoro di diverso tipo:
- Community Group e Interest Group, pensati per lo scambio di Idee e proposte.
- Working Group, che hanno il compito di produrre documentazione e standard.

1.2. Web 2.0

Con questo termine non si intende un cambio di tecnologie ma bensì un uso diverso di queste, il web infatti diventa una piattaforma fortemente orientata al mash-up di dati da sorgenti diverse, su cui pubblicare contenuti ed elaborare dati con facilità.
Le applicazioni web diventano sempre più veloci, complesse ed interattive.

I fattori determinanti per questo cambiamento sono stati:

- Enorme diffusione dei dispositivi mobili
- Avanzamento tecnologico di questi dispositivi che ormai hanno capacità avanzate di memorie e elaborazione
- Accesso ad internet diffuso e (ultra) veloce

Mentre per quanto riguarda le tecnologie Web, due aspetti ortogonali sono:

- Affermazione del paradigma REST per sfruttare le caratteristiche di HTTP e creare applicazioni solide e scalabili
- Sviluppo di linguaggi di programmazione client-side e di AJAX, che permette richieste asincrone al server e pre-caricamento di dati

Così le pagine web diventano la base per applicazioni che aggregano contenuti, anche da sorgenti diverse e in modo sofisticato, esistono molti servizi che producono solamente dati che poi verranno elaborati dai client. Diventa così sempre più importante fornire i propri dati in modo chiaro attraverso un'API (Application Programming Interface).

1.3. API Web

Così come per i linguaggi di programmazione, un'API Web definisce le modalità per interagire con un'applicazione. In pratica, elenca le possibili richieste e risposte che l'applicazione è in grado di gestire. Le applicazioni sia server-side che client-side possono quindi raccogliere i dati per poi elaborarli invocando una o più API.

- **Esistono applicazioni diverse costruite sugli stessi dati**
- **Applicazioni di mash-up che combinano dati in formati diversi e da sorgenti diverse.**

Dal web of content al web of data
I contenuti web sono letti non solo da essere umani ma sempre più da agenti software, questo succedeva anche in passato ma il processo era molto più complicato visto che le API erano poco diffuse.
Il web diventa quindi una piattaforma di dati intercollegati e non solo di contenuti ipertestuali, da link semplici si passa a link complessi e di diverso tipo in grado quindi di esprimere relazioni tra oggetti e costruire una base di conoscenza distribuita.

1.4. Conclusioni

Il WWW ha una storia relativamente recente ma ha visto diverse *evoluzioni ed involuzioni.* Da un sistema di contenuti ipertestuali si è arrivati ad un sistema di dati e applicazioni, con modalità di accesso sempre più variegate. Le tecnologie di base tuttavia sono sostanzialmente le stesse.

2. Uniform Resource Identifier

Gli URI sono una sintassi usata in WWW per definire i nomi e gli indirizzi delle risorse, sono

stati verosimilmente il fattore determinante del World Wide Web. Attraverso questi strumenti il WWW è stato in grado di identificare risorse accessibili tramite il proprio protocollo, HTTP, e tramite tutti gli altri protocolli esistenti.

Il punto principale a cui gli altri sistemi non erano arrivati era una sintassi universale, indipendente dal protocollo e facilmente memorizzabile (o quasi) con cui identificare le risorse di rete.

2.1. URI. URL e URN

Gli Uniform Resource Identifier sono, per definizione:

- Uniform Resource Locator (URL): una sintassi che contiene informazioni immediatamente utilizzabili per accedere alla risorsa (ad esempio, il suo indirizzo di rete).
- Uniform Resource Names (URN): una sintassi che permette una etichettatura permanente e non ripudiabile della risorsa, indipendentemente dal riportare informazioni sull'accesso. Necessario quindi un meccanismo di traduzione verso gli URL.

2.2. Risorsa vs. File

Una risorsa non è necessariamente un file presente su un filesystem ma potrebbe essere:

- In un database, e l'URI essere la chiave di ricerca
- Il risultato dell'elaborazione di un'applicazione, e l'URI essere i parametri di elaborazione.
- Una risorsa non elettronica (un libro, una persona…) e l'URI essere il suo nome, es. account Twitter, Instagram
- Un concetto astratto.

Per questo si usa il termine Risorsa, invece che File, e si fornisce una sintassi indipendente dal sistema effettivo di memorizzazione.

2.3. Organizzazione degli URI

Gli URI sono progettati per fornire **spazi di nomi organizzati gerarchicamente:**

```
URI = schema :  [II authority] path [? query] [# fragment]
```

- Lo schema (negli URL è il protocollo) è identificato da una stringa registrata presso IANA usata come prefisso. L'autorità è a sua volta divisa in:
  - `authority = [userinfo @] host [: port]`
    - La parte userInfo non deve essere presente se lo schema non prevede identificazione personale.
    - La parte host è o un nome di dominio o un indirizzo IR. La port può essere omessa se ci si riferisce ad una well-known port (per http la 80).
- La parte path è la parte identificativa della risorsa all'interno dello spazio di nomi identificato dallo schema e (se esistente) della authority. Questa parte è suddivisa in blocchi tramite il carattere /, ciascuno dei quali è un componente del path organizzato in gerarchia.

- La parte query individua un'ulteriore specificazione della risorsa all'interno dello spazio di nomi identificato dallo schema e dall'URI precedente. Solitamente sono parametri passati all'URI per specificare un risultato dinamico. La forma tipica:
  - `nome1=valore&nome2=valore+in+molte+parole`
- La parte fragment individua una risorsa secondaria (una risorsa associata, dipendente in molti casi un frammento della risorsa primaria). Viene definita dopo la keyword #.

HTTP è lo schema più usato negli URI, mentre HTTPS prevede la cifratura dei messaggi in entrambi i versi. Per il resto è identico a HTTP:

```
http[s] ://host[:port]/path[?query][#fragment]
```

Note:

- host è l'indirizzo TCP-IP o DNS della macchina su cui si trova la risorsa.
- port è la porta a cui il server è in ascolto per le connessioni. Di default è 80 per HTTP e 443 per HTTPS.

2.4. Lo schema FILE (RFC 8089)

Da accesso ai file di un file system locale (cioè del computer su cui gira il browser). Non girano applicazioni server-side, quindi nessuna connessione HTTP. La sintassi consiste in:

```
file://<host>/<path> [#fragment]
```

La parte host può essere eliminata, assumendo che sia localhost, che porta alla sintassi più frequente:

```
file://localhost/path
```

```
file:///c:/Users/filippo/Pictures/img1.jpg
```

2.5. Lo schema DATA (RFC 2397)

Uno schema non gerarchico, che non fa riferimento ad una risorsa, ma contiene la risorsa: tutti i dati della risorsa sono inseriti nell'URI vero e proprio. Viene usato maggiormente per immagini inline su cui non si vuole attivare una connessione HTTP separata. La sintassi è:

```
data:[<media type>][;base64],<data>
```

- Media type è un media type MINE (*paragrafo su MINE e base64*).
- Data sono i dati codificati con il media type appena indicato (*base64*).

2.6. Routing e URL assoluti e relativi

Un web server è un software in grado di gestire richieste HTTP. E' in ascolto su una determinata porta TCP/IP.

Una route è un'associazione della parte *path* di un uri ad una risorsa gestita o restituita da un server web.

- Managed route: il server associa ogni URI ad una risorsa attraverso il file system locale

(risorse statiche) oppure generate attraverso una computazione (risorse dinamiche).
    - Molto di moda oggi con NodeJS e Espress.
- File-system route: il server associa la radice della parte path ad una directory del file system locale e ogni filename valido all'interno di quella directory genera un URI corretto e funzionante.
    - Il vecchio approccio via web server come Apache.

Codice di una managed route:

```
var router = require("express").Router();

function getName(req, res) { res.send("<p>Bob</p>"); }

function getEmail(req, res) {res.send("bob@unibo.it");}

router.get("/name", getName);

router.get("/email", getEmail);

app.use("/css", express.static('css'));
```

Risultato:

```
http://www.mydomain.com/name - <p>Bob</p>

http://www.mydomain.com/email - bob@unibo.it

http://www.example.com/css/style.css - css/style.css
```

2.7. URI reference (o URI relativo)

Un URI assoluto contiene tutte le parti predefinite dal suo schema, esplicitamente precisate. Un URI gerarchico può però anche essere relativo, ed in questo caso riportare solo una parte dell'URI assoluto corrispondente "tagliando le parti a sinistra".

Un URI reference fa sempre riferimento ad un URI di base rispetto al quale fornisce porzioni differenti.

- Es: L'URL reference `pippo.html` posto dentro al documento di URI `http://www.sito.com/dir1/dir2/pluto.html` fa riferimento al documento il cui URI assoluto è `http://www.sito.com/dir1/dir2/pippo.html`

Risolvere un URI relativo: (dato l'URI di base - `http://www.site.com/dir1/doc1.html`*)*

- `#anchor1`: si risolve come: `http://www.sito.com/dir1/doc1.html`. Questo perché inizia con "#", di conseguenza è un frammento interno allo stesso documento di base.
- `http://www.sito.com/dir2/doc2.html`: si risolve come `http://www.sito.com/dir2/doc2.html` questo perché inizia con uno schema, quindi è un URI assoluto.
- `/dir3/doc3.html`: si risolve come `http://www.sito.com/dir3/doc3.html` questo perché se inizia con / allora è un path assoluto all'interno della stessa autorità del

documento di base, di conseguenza gli va applicata la stessa parte di autorità.
- `doc4.html`: si risolve come `http://www.sito.com/dir1/doc4.html` mentre `dir5/doc5.html` si risolve come `http://www.sito.com/dir1/dir5/doc5.html` questo perché si estrae il path assoluto dell'URI di base, meno l'ultimo elemento e si aggiunge in fondo l'URI relativo.
- `./doc6.html` si risolve come `http://www.sito.com/dir1/./doc6.html` che è equivalente a `http://www.sito.com/dir1/doc6.html`.
- `../doc7.html` si risolve come `http://www.sito.com/dir1/../doc7.html` che è equivalente a `http://www.sito.com/doc7.html`.

2.8. MINE - Multipurpose Internet Mail Extensions

SMTP è un protocollo *text-based,* per lo scambio di messaggi di posta elettronica e la verifica dei destinatari dei messaggi. Ci sono però alcuni limiti fondamentali che impediscono la trasmissione di file binari:

- La lunghezza massima del messaggio è 1 Mb.
- I caratteri accettati sono solo ASCII a 7 bit.
- Ogni messaggio deve contenere una sequenza CRLF ogni 1000 caratteri o meno.

MINE (Multipurpose Internet Mail Extensions) è stato definito per bypassare questi limiti e qui è rilevante perché permette di indicare la codifica e i tipi delle risorse scambiate via HTTP.

```
Mine-Version: 1.0

Content-Type: text/plain; charset="US-ASCII"

Content-Transfer-Encoding: 7bit
```

Dichiarazione tipo: Tutti i messaggi MIME vengono identificati da un Content Type, che definisce il tipo di dati del messaggio e aiuta l'applicazione ricevente a gestire il messaggio e a invocare l'applicazione più adatta.

Messaggi multi-tipo: Un messaggio MIME può contenere parti di tipo diverso (es. un messaggio di tipo testo e un attachment binario).In questo caso si creano dei sotto-messaggi MIME per ciascuna parte (con il suo bravo content-type) e il messaggio MIME complessivo diventa "multi-parte", qualificando e codificando in maniera diversa ciascuna sottoparte.

Vengono introdotti anche nuovi Header come `Content-Type` *e* `Content-Transfer-Encoding`.

3. HyperText Transfer Protocol

HTTP è un protocollo client-server, generico e stateless utilizzato non solo per lo scambio di documenti Web ma in molte applicazioni distribuite:

- Client-server: il client attiva la connessione e richiede dei servizi. Il server accetta la connessione, nel caso identifica il richiedente e risponde alla richiesta. Alla fine chiude la connessione.
- Generico: HTTP è indipendente dal formato di dati con cui vengono trasmesse le risorse.

Può funzionare per qualsiasi tipologia di file.
- Stateless: il server non è tenuto a mantenere informazioni che persistono tra una connessione e la successiva. Il client è tenuto a ricreare da zero il contesto necessario al server per rispondere.

HTTP permette lo scambio di risorse identificate da URI, inoltre separa nettamente la loro rappresentazione e fornisce meccanismi di negoziazione del formato di dati, cioè la possibilità di richiedere e ricevere (la rappresentazione di) una stessa risorsa in formati diversi.

Inoltre questo protocollo implementa sofisticate politiche di caching che permettono di memorizzare copie delle risorse sui server (proxy, gateway) coinvolti nella trasmissione e possono controllare in modo accurato la validità di queste copie.

3.1. Connessione HTTP

Una connessione HTTP è composta da una serie di richieste ed una serie corrispondente di risposte, le connessioni sono persistenti con:

- `Pipelining`: trasmissione di più richieste senza attendere l'arrivo della risposta alle richieste precedenti. Ma le risposte sono restituite nello stesso ordine delle richieste.
- `Multiplexing`: nella stessa connessione è possibile avere richieste e risposte multiple, restituite anche in ordine diverso rispetto alle richieste e "ricostruite" nel client.

Con la versione 2 di HTTP sono state introdotte molte novità per migliorare le performance, tra cui:

- Operazioni PUSH: Il server anticipa le richieste successive del client.
- Compressione degli Header: I dati non sono trasmessi in chiaro ma compressi (e in parallelo).

IMMAGINE CONNESSIONI HTTP - 03a-HTTP.pdf SLIDE 5

3.2. Richieste HTTP

Una richiesta HTTP si compone di diversi elementi:

- `Method`: consiste nell'azione del server richiesta dal client.
- `URI`: identificativo della risorsa locale al server.
- `Version`: numero di versione di HTTP.
- `Header`: sono linee RFC822 divise in:
  - `Header generali`
  - `Header di entità`
  - `Header di richiesta`
- `Body`: è un messaggio MINE.

esempio richiesta HTTP:

```
GET /beta.html HTTP/1.1

Referer: http://www.alpha.com/alpha.html
```

```
Connection: Keep-Alive

User-Agent: Mozilla/4.61 (Macintosh; I; PPC)

Host: www.alpha.com:80

Accept: image/gif, image/jpeg, image/png, */*

Accept-Encoding: gzip

Accept-Language: en

Accept-Charset: iso-8859-1,*,utf-8
```

3.3. Metodi HTTP

I metodi indicano l'azione che il client richiede al server sulla copia della rappresentazione della risorsa. Un uso corretto di questi metodi aiuta a creare applicazioni in grado di sfruttare al meglio i meccanismi di caching di HTTP.

I principali metodi sono:

- `GET`
- `HEAD`
- `POST`
- `PUT`
- `DELETE`
- `OPTIONS`
- `PATCH`

I metodi possiedono due proprietà fondamentali:

- `Sicurezza`: un metodo è sicuro se non genera cambiamenti allo stato interno del server. Un metodo sicuro può essere eseguito da un nodo intermedio (una cache) senza effetti negativi.
- `Idempotenza`: un metodo è idempotente se l'effetto sul server di più richieste identiche è lo stesso di quello di una sola richiesta. Un metodo di questo tipo può essere ri-eseguito da più agenti o in più tempo diversi senza effetti negativi.

Metodo GET

Il più importante metodo HTTP è il GET, che richiede una risorsa ad un server, viene considerato il metodo più frequente. Questo metodo è sicuro ed idempotente.

```
GET /courses/tw.html

GET /students/123456

GET /students/123456/exams/
```

Metodo HEAD

Il metodo HEAD è simile al GET, ma il server deve rispondere soltanto con gli header relativi,

senza il corpo. Viene usato per la verifica della validità, accessibilità e coerenza in cache di un URI. Anche HEAD è sicuro ed idempotente.

```
HEAD /courses/tw.html
```

Metodo POST

Il metodo POST serve per trasmettere informazioni dal client al server relative alla risorsa identificata nell'URI. Può essere usato anche per creare nuove risorse oppure per inviare dati di un form ad una applicazione server-side. POST non è sicuro né idempotente.

```
POST /courses/1678

{

 "titolo":"Tecnologie Web",

 "descrizione":"Il corso..bla..bla.."

}
```

Metodo PUT

Il metodo PUT serve per trasmettere delle informazioni dal client al server, creando o sostituendo la risorsa specificata. Non offre garanzie di controllo degli accessi o locking, PUT è idempotente ma non sicuro.

```
PUT /courses/1678

{

 "id":1678,

 "titolo":"Tecnologie Web",

}
```

Metodo DELETE

Il metodo DELETE serve per rimuovere le informazioni connesse con una risorsa. Dopo l'esecuzione di un DELETE la risorsa non esiste e ad ogni GET di quella determinata risorsa verrà ritornato un errore `404 Not Found`. Il metodo DELETE è idempotente e non sicuro.

```
DELETE /courses/1678
```

Metodo PATCH

Il metodo PATCH è usato per aggiornare parzialmente la risorsa identificata dall'URI, viene usato per indicare quindi modifiche incrementali e non sovrascrivere risorse dal server. Indica quindi le modifiche da effettuare su una risorsa esistente. Questo metodo è né sicuro né idempotente.

```
PATCH /courses/1678
```

```
{
 "op" : "update"
 "cfu":"6"
}
```

Metodo OPTIONS

Questo metodo viene usato per verificare opzioni, requisiti di un server, senza implicare che eseguirà poi un'altra richiesta. Viene usato per il problema del *cross-site scripting:* CORS.

Riassunto

Metodo HTTP	IDEMPOTENZA	SICUREZZA
GET	SI	SI
HEAD	SI	SI
OPTION	SI	SI
DELETE	SI	NO
PUT	SI	NO
PATCH	NO	NO
POST	NO	NO

3.4. Risposta HTTP

La risposta HTTP si compone dei seguenti elementi:

- `Status Code`: indica se la richiesta è andata a buon fine o meno
- `Version`: numero di versione di HTTP
- `Header`: come per la richiesta:
  - `Header generali`
  - `Header di entità`
  - `Header di risposta`
- `Body`: un messaggio MIME

esempio risposta HTTP:

```
GET /index.html HTTP/1.1
Host: www.cs.unibo.it:80
————————————————————————
HTTP/1.1 200 OK
```

```
Date: Fri, 26 Nov 2007 11:46:53 GMT

Server: Apache/1.3.3 (Unix)

Last-Modified: Mon, 12 Jul 2007 12:55:37 GMT

Accept-Ranges: bytes

Content-Length: 3357

Content-Type: text/html

<HTML> …. </HTML>
```

Status code

Lo status code è un numero di tre cifre, di cui la prima indica la classe della risposta, e le altre due la risposta specifica. Le classi si suddividono in:

- `1xx: Informational`: una risposta temporanea alla richiesta, durante il suo svolgimento.
- `2xx: Successful`: il server ha ricevuto, capito e accettato la richiesta.
- `3xx: Redirection`: la richiesta è corretta, ma sono necessarie altre azioni da parte del client per portare a termine la richiesta.
- `4xx: Client error`: La richiesta del client non può essere soddisfatta per un errore da parte del client.
- `5xx: Server error`: la richiesta può anche essere corretta, ma il server non riesce a soddisfarla per un problema interno.

esempi di status code:

```
100 Continue (se il client non ha ancora mandato il body)

200 Ok (GET con successo)

201 Created (PUT con successo)

301 Moved permanently (URL non valida, il server conosce la nuova
posizione)

400 Bad request (errore sintattico nella richiesta)

401 Unauthorized (manca l'autorizzazione)

403 Forbidden (richiesta non autorizzabile)

404 Not found (URL errato)

500 Internal server error (tipicamente un errore nel codice
server-side)
```

L'uso corretto degli status code aiuta a costruire API chiare e più semplici da usare, il client non ha la necessità di leggere il body della risposta già dallo status code può capire se la richiesta è

andata a buon fine. Inoltre permette a tutte le entità coinvolte di capire cosa sta succedendo, e sfruttare i meccanismi di caching e redirezione di HTTP.

3.5. Gli Header

Gli header sono righe di testo (RFC822) che specificano informazioni aggiuntive, sono presenti sia nelle richieste che nelle risposte e ne descrivono diversi aspetti:

- `Header generali`: informazioni sulla trasmissione.
- `Header di entità`: informazioni sulla risorsa e i dati trasmessi.
- `Header di richiesta`: Informazioni sulla richiesta effettuata o sulla risposta generata.

Header generali

Questa tipologia di header si applica solo ai messi trasmessi da una richiesta o una risposta, ma non necessariamente alla risorsa trasmessa. Alcuni esempi possono essere:

- `Date`: data e ora della trasmissione.
- `Transfer-Encoding`: il tipo di formato di codifica usato per la trasmissione.
- `Cache-Control`: il tipo di meccanismo di caching richiesto o suggerito per la risorsa.
- `Connection`: il tipo di connessione da usare (se tenerla attiva o chiuderla dopo la risposta).

Header dell'entità

Gli header dell'entità danno informazioni sul body del messaggio, o, se non vi è body, sulla risorsa specificata. Alcuni esempi possono essere:

- `Content-Type`: il tipo MIME dell'entità del body. Questo header è obbligatorio in ogni messaggio che abbia un body.
- `Content-Length`: la lunghezza in byte del body, obbligatorio soprattutto se la connessione è persistente.
- `Content-Encoding, Content-Language, Content-Location, Content-MD5, Content-Range`: la codifica, il linguaggio, l'URL della risorsa specifica, il valore di digest MD5 e il range richiesto della risorsa.

Header della richiesta

Gli header della richiesta sono posti dal client per specificare informazioni sulla richiesta e su se stesso al server. Alcuni esempi possono essere:

- `User-Agent`: una stringa che descrive il client che origina la richiesta, tipo, versione e sistema operativo del client (tipicamente).
- `Referer`: l'URL della pagina mostrata all'utente mentre richiede il nuovo URL.
- `Host`: il nome di dominio e la porta a cui viene fatta la connessione.

Header della risposta

Gli header della risposta sono posti dal server per specificare informazioni sulla risposta e su se

stesso al client. Alcuni esempi:

- `Server:` una stringa che descrive il server: tipo, sistema operativo e versione.
- `WWW-Authenticate:` challenge utilizzata per i meccanismi di autenticazione.

L'importanza del content type

Tra gli header della risposta sono particolarmente utili quelli che definiscono il tipo di dato contenuto nella risposta, queste informazioni permettono al client di processare correttamente la risorsa. Se viene fornita una risposta gli `header Content-Type e Content-Length` sono obbligatori.

4. API Rest

Un API Web descrive un'interfaccia HTTP che permette ad applicazioni remote di utilizzare i servizi dell'applicazione, che possono essere:

- Applicazioni automatiche che utilizzano i dati dell'applicazione.
- Applicazioni Web che mostrano all'utente un menù di opzioni, magari anche un form, e gli permettono di eseguire un'azione sui dati dell'applicazione.

4.1. Rest

REST è l'acronimo di *REpresentational State Transfer,* ed è il modello architetturale che sta dietro al WWW e in generale alle applicazioni web "ben fatte" secondo i progettisti di HTTP.

L'obiettivo è creare API consistenti, prevedibili e facili da capire ed utilizzare.

Con il modello CRUD si intende il pattern tipico delle applicazioni di trattamento dei dati, si ipotizza che tutte le operazioni sui dati siano una di:

- `Create`: inserimento nel database di un nuovo record.
- `Read`: accesso in lettura al database, che può essere individuale (risposta con una sola riga), oppure contenitore (risposta con più righe).
- `Update`: aggiorna nel database una o più righe.
- `Delete`: rimuove dal database una o più righe.

L'architettura REST si basa su quattro punti fondamentali:

- Definire **risorsa** ogni concetto rilevante dell'applicazione Web.
- Associargli un **URI** come identificatore e selettore primario.
- Usare i verbi HTTP per esprimere ogni **operazione** dell'applicazione secondo il modello CRUD:
  - creazione di un nuovo oggetto (metodo `POST`)
  - visualizzazione dello stato della risorsa (metodo `GET`)
  - cambio dello stato della risorsa (metodo `PUT`)
  - cancellazione di una risorsa (metodo `DELETE`)
- Esprimere in maniera parametrica ogni **rappresentazione dello stato interno della risorsa**, personalizzabile dal richiedente attraverso un **Content Type** preciso.

REST identifica due concetti fondamentali: **individui** e **collezioni**.

es: un cliente o l'insieme di tutti i clienti.
es: un esame o l'insieme di tutti gli esami.

Le collezioni possono "contenere" individui o altre collezioni, è consigliabile strutturare gli URI in modo gerarchico, per esplicitare queste relazioni.

es:
- tutti i clienti: `/clients/`
- il cliente 1234: `/clients/1234`
- tutti gli ordini del cliente 1234: `/clients/1234/orders/`

4.2. Linee guida degli URI in REST

Le collezioni sono intrinsecamente plurali, mentre gli individui sono intrinsecamente singolari. Le collezioni devono essere visivamente distinguibili dagli individui. Per questo si usa un termine plurale e uno slash in fondo all'URI.

URI	**Rappresentazione**
`/customers/`	collezione dei clienti
`/customers/abc123`	cliente con id=abc123
`/customers/abc123/`	collezione delle sotto-risorse del client con id=abc123 (es. indirizzi, telefoni, ecc)
`/customers/abc123/addresses/1`	primo indirizzo del cliente con id=abc123

Un filtro genera un **sottoinsieme** specificato attraverso una regola di qualche tipo. La gerarchia permette di specificare i tipi più frequenti e rilevanti di filtro. Altrimenti si sa la parte di query dell'URI di una collezione:

URI	**Rappresentazione**
`/regions/ER/customers/`	collezione dei clienti dell'Emilia Romagna
`/status/active/customers/`	collezione dei clienti attivi
`/customers/?tel=0511234567 o /customers/?search=tel&value=0512 34567`	collezione dei clienti che hanno telefono = 051 1234567
`/customers/?search=sales&value=10 000&op=gt`	collezione dei clienti che hanno comprato più di 10000

esempi uso dei verbi HTTP in REST:

- `GET /customers/`: elencare tutti i clienti
- `GET /customers/abc123`: accedere ai dati del cliente id=abc123
- `POST /customers/`: creare un nuovo cliente (il client non decide l'identificatore)
- `PUT /customers/abc123`: creare un nuovo cliente (il client decide l'identificatore)
- `PUT /customers/abc123`: modifica tutti i dati del cliente ad=abc123
- `POST /customers/abc123/telephones/`: modifica alcuni dati del cliente id=abc123
- `DELETE /customers//abc123`: cancella il cliente id=abc123

Dopo la modifica del documento di HTTP (RCF 7231) del 2014 viene specificato che il metodo POST può essere usato per un ampio specchio di situazioni, purché non sovrapposta a quella degli altri verbi.

4.3. Descrivere una RESTful API

Una API è RESTful se utilizza i principi REST nel fornire accesso ai servizi che offre, per documentare un API è necessario definire:

- `End-point (URI / Route)` che supporta sia collezioni che elementi singoli.
- Metodi HTTP di accesso (`GET, POST, PUT, DELETE`…)
- Rappresentazioni di `Input` e `Output`, di solito non si una un linguaggio di schema ma un esempio fittizio e sufficientemente complesso.
- Condizioni di errore e i messaggi che restituisce in questi casi.

Swagger e Open API

Swagger è un ecosistema di tool per la creazione, costruzione, documentazione e accesso ad un API soprattutto in ambito REST. In particolare ha creato un linguaggio per la documentazione, modifica e test di API REST. Nel 2016 il linguaggio è stato reso pubblico ed è diventato Open API, può essere serializzato sia in JSON che in YAML.

Descrizione sintassi:

- `Sezione path`: la parte centrale di un'API descrive i percorsi (URL) corrispondenti alle operazioni possibili sull'API, la struttura è descritta come `<host>/<basePath>/<path>`, per ogni percorso (path o endpoint) si definiscono tutte le possibili operazioni che, secondo i principi rest, sono identificate dal metodo HTTP corrispondente, per ogni path quindi ci sono tante sottosezioni quante sono le operazioni per ognuna:
  - informazioni generali
  - parametri di input / output

struttura di un path:

```
/pet/{petId}:
   get:
      summary: "Find pet by ID"
```

```
      description: "Returns a single pet"
      operationId: "getPetById"
      produces:
         - "application/xml"
         - "application/json"
      parameters:
      …
      …

   post:
      summary: "Updates a pet in the store with form data"
      description: ""
      operationId: "updatePetWithForm"
      parameters:
      …
      …
```

- `Parametri di Input`: sono descritti nella sezione `parameters` e per ogni parametro è possibile definire:
  - tipo del parametro: keyword in che può assumere valori: `path, query o body`
  - nome (`name`) e descrizione (`description`)
  - se è opzionale o obbligatorio (`required`)
  - formato del/i valore/i che il dato può assumere (`schema`)

struttura di un parametro path `/pet/{petId}`*:*

```
      parameters:
         - name: petId
           in: path
           description: ID of the pet to return
           required: true
           type: integer
           format: int64
```

struttura di un parametro query `/pet/`*: /pet/?status=ready*

```
      parameters:
         - name: status
           in: query
           description: Status value that need to be considered
           required: true
           type: array
           items:
              type: string
```

struttura di un parametro body `/user/{username}: Oggetto <User> nel body`

```
      parameters:
         - name: username
           in: path
           description: Name that need to be updated
```

```
          required: true
          type: string
        - name: body
          in: body
          description: Updated user object
          required: true
          schema:
             $ref: "#/definitions/User" // Definizione dell'oggetto
```

La sezione `definitions` permette di definire i tipi degli oggetti, le loro proprietà e possibili valori. Per accedere a queste tipologie di dato si utilizza `schema -> $ref` sia per le richieste sia per le risposte.

esempio di schema

```
User:
   type: object
   properties:
      id:
         type: integer
         format: int64
      username:
         type: string
      firstName:
         type: string
      lastName:
         type: string
      userStatus:
         type: integer
         format: int32
         description: User Status
Oder:
   type: object
   properties:
      id:
         type: integer
         format: int64
      petId:
         type: integer
         format: int64
      status:
         type: string
         description: Order Status
         enum:
            - waiting
            - approved
            - delivered
```

L'output (dati, codici e messaggi di errore) sono definiti attraverso la keyword `responses`, serve per specificare che tipologia di output è atteso nel body della risposta. Inoltre ogni risposta ha un ID numerico univoco, associato al codice HTTP corrispondente:

- `200` viene usato per indicare che non c'è stato alcun errore
- da `400` in su vengono, in genere, usati per indicare messaggi di errore

```
parameters
// ALL THE PARAMS LIST
responses:
   "200":
      description: successful operation
      schema:
         type: array
         items:
            $ref: "#/definitions/Pet"
   "400":
      description: invalid status value
```

editor swagger online https://editor.swagger.io/

5. Markup: SGML, XML, DOM e (X)HTML

Il sorgente delle pagine Web è in HTML (HyperText Markup Language), un linguaggio che "marca":

- struttura del documento
- informazioni di presentazione
- collegamenti ipertestuali (e URL della destinazione)
- risorse multimediali (e URL)

Il browser è in grado di interpretare queste informazioni e visualizzare la pagina finale. Con markup si intende ogni mezzo per rendere esplicita una particolare interpretazione di un testo. Oltre a rendere il testo più leggibile, il markup permette anche di specificare ulteriori usi del testo. Esistono diversi tipi di markup:

- `Presentazionale`: indica effetti (grafici e altro) per rendere più chiara la presentazione del contenuto.
- `Procedurale`: indica ad un sistema automatico che procedura (serie di istruzioni) eseguire per visualizzare il contenuto.
- `Descrittivo`: individua strutturalmente il tipo di ogni elemento del contenuto e ne evidenzia il ruolo.
- `Referenziale`: consiste nel fare riferimento ad entità esterne al documento per fornire significato o effetto grafico ad elementi del DOM.

Un meta-linguaggio è un linguaggio per definire i linguaggi, una grammatica di costruzione di linguaggi. Infatti SGML e XML non sono linguaggi di markup, ma linguaggi con cui definiamo i linguaggi di markup, quindi metalinguaggio di markup. Questi due metalinguaggi, per esempio, non sanno cos'è un paragrafo, una lista, un titolo, ma forniscono le grammatiche che ci permettono di definirli.

HTML nasce come linguaggio di markup definito usando la grammatica SGML, il markup in SGML è:

- **strutturato**: possibile definire una serie di regole affinché il testo sia considerabile strutturalmente corretto.
- **gerarchico**: gli elementi del testo possono essere annidati e specificare la struttura in maniera gerarchica.

Un documento con markup di derivazione SGML - inclusi HTML e XML - contiene:

- Elementi, attributi e testo
- Entità
- Commenti e Processing Instructions

Con elementi si intendono le parti di documento dotate di un senso proprio e sono individuate da tag iniziale, contenuto e tag finale. Mentre con il termine attributi ci si riferisce alle informazioni aggiuntive sugli elementi, tipicamente espressi in sintassi `nome="valore"`. Infine il testo è il contenuto vero e proprio, viene anche detto `#PCDATA` (Parsed Character DATA), perché è processato per sostituire le entità.

Le entità invece sono frammenti di documento memorizzati separatamente e richiamabili all'interno del documento, HTML, per esempio, definisce un set di entità utilizzabili per caratteri non ammessi nella codifica usata (es. lettere accentate in ASCII).

I documenti di markup possono contenere commenti, che non fanno parte del contenuto, che vengono ignorati dalle applicazioni di rendering. Mentre le Processing Instructions (PI) sono elementi particolari usati per dare indicazioni alle applicazioni, sono molto usate in XML, poco in HTML.

5.1. DOM

Se un documento è ben formato può essere rappresentato in memoria come un albero, chiamato DOM, il Document Object Model è un'interfaccia di programmazione (API) per documenti sia XML che HTML.

Definisce la struttura logica dei documenti ed il modo in cui si accede e si manipola un documento, utilizzando questo strumento i programmatori possono costruire documenti, navigare attraverso la loro struttura, e aggiungere, modificare o cancellare elementi.

L'oggetto principale di DOM è DOMNode, che però è un'interfaccia, che quindi viene utilizzata solamente per creare classi. Il core del DOM definisce alcune classi fondamentali per i documenti HTML e XML, e ne specifica proprietà e metodi.

Gli oggetti principali definiti nel DOM sono:

- `DOMDocument`: il documento di cui si sta parlando
- `DOMElement`: ogni singolo elemento del documento
- `DOMAttr`: ogni singolo attributo del documento
- `DOMText`: ogni singolo nodo di testo del documento

esempio di `DOMNode`:

Questo elemento specifica i metodi per accedere a tutti gli elementi di un nodo di un documento,

inclusi il nodo radice, il nodo documento, i nodi elemento, i nodi attributo, i nodi testo, ecc.

Semplificando:

Membri	Metodi
`nodeName`	`insertBefore()`
`nodeValue`	`replaceChild()`
`nodeType`	`removeChild()`
`parentNode`	`appendChild()`
`childNodes`	`hasChildNodes()`
`attributes`	`hasAttributes()`

Le versioni di HTML hanno adottato in modo molto diverso le regole di buona forma e validazione, sotto le diverse spinte del W3C e dei produttori di browser. Esistono **differenze** anche **sensibili** tra un documento **HTML corretto** e un documento **HTML visualizzabile** da un browser Web.

Non è detto che un documento HTML “accettabile” sia “ben fatto”.

5.2. HTML HyperText Markup Language

Sono da considerare alcune premesse sintattiche, infatti HTML non è sensibile al maiuscolo / minuscolo, inoltre collassa tutti i caratteri si whitespace (SPACE, TAB…) in un unico spazio. Questo permette di organizzare il sorgente in modalità leggibile senza influenzare la visualizzazione su browser. L'entità ` ` permette di inserire spazi non collassabili.

HTML definisce un certo numero di entità per quei caratteri che sono:

- proibiti perché usati in HTML (`<`,`>`,`&`,`"`)
- proibiti perché non presenti `nell'ASCII` a 7 bit

Sono definiti anche degli attributi globali ovvero quelli definiti su tutti gli elementi del linguaggio HTML. I `coreattrs` costituiscono attributi di qualificazione e associazione globale degli elementi. Per lo più per CSS e link ipertestuali:

- `id`: un identificativo univoco (su tutti il documento)
- `style`: un breve stile CSS associato al singolo elemento
- `class`: una lista (separata da spazi) di nomi di classe
- `title`: un testo secondario associato all’elemento (per accessibilità e informazioni aggiuntive)

Gli attributi globali `i18n` (internationalization) garantiscono l’internazionalizzazione del linguaggio e la coesistenza di script diversi:

- `lang`: una codifica dei linguaggi umani (stringa di due caratteri: it, en, fr…).
- `dir`: uno dei due valori ltr (left to right) o rlt (right to left) per indicare la direzione di flusso secondario del testo.

Mentre gli attributi di evento permettono di associare script a particolari azioni sul documento e sui suoi elementi:

- `onclick, ondoubleclick, onmouseover, onkeypress,` ecc.

Gli elementi di HTML sono organizzati secondo alcune categorie:

- Struttura complessiva (`HTML, HEAD, BODY`)
- Struttura del contenuto (`SECTION, NAV, FOOTER`)
- Elementi di blocco e liste (`P, H1, H2…, DIV, UL, OL…`)
- Elementi inline (`B, I, SPAN`)
- Elementi speciali (`A, IMG, HR, BR`)
- Tabelle (`TABLE, TR, TD, TH`)
- Form (`FORM, SELECT, OPTION, INPUT`)

vedere codifica di ogni elemento su https://www.w3schools.com/html/

5.3. Caratteri ammessi negli URI

I caratteri degli URI possono essere:

- **Unreserved:** utilizzabili liberamente, sono tutti caratteri alfanumerici e alcuni caratteri di punteggiatura non ambigui ovvero: `- _ ! * ' ( )`
- **Reserved:** caratteri che hanno delle funzioni particolari in uno o più schemi di URI. Devono essere utilizzati per le loro funzioni e sono: `; / ? : @ & = + $`
- **Escaped:** caratteri riservati di cui si fa *escaping* per usarli nelle stringhe identificative.

5.3.1. Caratteri escaped

I caratteri escaped fanno riferimento alle seguenti tipologie di caratteri:

- I caratteri non US-ASCII
- I caratteri di controllo: US-ASCII < 32
- I caratteri unwise: `{ } | \ ^ [ ] ` `
- I delimitatori: spazio `< > # % "`
- I caratteri riservati quando usati in contesto diverso dal loro uso riservato

In questo caso i caratteri vanno posti in maniera escaped, secondo la seguente sintassi:

- `%XY` dove `XY` è il codice esadecimale del carattere

Esempi:

- `http://www.alpha.edu/a/b/c/d`
- `http://www.alpha.edu/a/b/c%2Fd`

I due link non sono uguali, perché, nonostante il codice esadecimale corrisponda al carattere corretto ,/, nel primo caso esso ha un significato gerarchico mentre nel secondo fa parte dell'ultima sottoparte della gerarchia: `c/d`.

5.3.2. Caratteri ammessi negli URI

La codifica `application/x-www-form-urlencoded` é usata nelle operazioni POST derivanti da un `form HTML`.

`http://www.site.com/serverside.py?name=John&surname=Smith&gender=m…`

- I codici non alfanumerici sono sostituiti da `%HH` con `HH` corrispondente al codice esadecimale del carattere.
- Gli spazi sono sostituiti da un +.
- I nomi dei controlli sono separati dal carattere `&`.
- Il valore é separato dal nome da =.

6. Cascading Style Sheet

Con questo formato è prevista ed incoraggiata la presenza di fogli di stile multipli, che agiscono uno dopo l'altro, in cascata, per indicare le caratteristiche tipografiche e di layout di un documento `HTML`.

- Controllo sia dell'autore che del lettore di un documento `HTML`
- Indipendente dalla specifica collezione di elementi ed attributi `HTML` (si può utilizzare anche con `XML`)

Il formato `HTML` prevede l'uso di stili `CSS` in tre modi diversi:

- Posizionato direttamente sull'elemento (sconsigliato)
- Posizionato nell'elemento `<style>`
- Posizionato nell'elemento `<link>`

Esistono tre modi principali di assegnare gli stili agli elementi:

- A tutti gli elementi di un certo tipo: con l'utilizzo del nome dell'elemento
- Tramite la specificazione di una categoria, chiamata comunemente `class`
- Tramite la specificazione di un attributo identificatore, chiamato `id`

Gli attributi di un elemento vengono presi non da uno (il primo, l'ultimo…) dei fogli di stile, ma composti dinamicamente sulla base del contributo di tutti, in cascata.

Esempio:

```
p { font-family: 'Geist Mono'; font-size: 12px; }

p { color: red; font-size: 11px; }

p { margin-left: 15px; color: green }
```

Gli attributi dell'elemento `p` saranno equivalenti a:

```
p {
      font-family: 'Geist Mono';
      font-size: 11px;
      margin-left: 15px;
      color: green;
}
```

Gli elementi HTML (e i contenitori CSS) sono organizzati in una struttura gerarchica. A parte alcune eccezioni, le proprietà CSS degli elementi sono ereditate, ossia assumono lo stesso valore che hanno nel contenitore dell'elemento a cui si riferisce la proprietà.

```
<p id="intro" style="color: red">
      Paragrafo su <span class="Place">Roma</span>
</p>
```

La proprietà display (che serve per specificare il flusso del testo, es blocco e inline) NON è ereditaria.

Nell'esempio sopra l'elemento `span` rimane inline, anche se contenuto in un altro blocco.

6.1.1. Computed Style

Lo stile finale di un elemento viene quindi "calcolato" combinando le diverse sorgenti e regole, viene applicato un algoritmo di ordinamento sulle dichiarazioni secondo alcuni principi:

- Tipo di dispositivo
- Importanza di una dichiarazione, esprimibile tramite il simbolo **!** prima di una regola
- Origine della dichiarazione (utente > autore > user agent)
- Specificità della dichiarazione (es. ID > class, specifica su un elemento > ereditarietà)
- Ordine in cui si trovano le dichiarazioni (si applica all'ultima)

6.1.2. Selettori principali

Un selettore permette di specificare un elemento o una classe di elementi dell'albero HTML al fine di associare caratteristiche CSS:

- `h1, p.codice, img[alt]`

Una regola è un blocco di statement associati ad un elemento attraverso l'uso di un selettore, i principali selettori sono:

Pattern	Significato	Descrizione	Esempio
`*`	Tutti gli elementi	Selettore universale	`*`
`E`	Un elemento di tipo E	Selettore di tipo	`h1`
`E.nomeClasse`	Un elemento di tipo E e di classe nomeClasse	Selettore di classe	`p.titolo`
`E#ilMioID`	Un elemento di tipo E con ID ilMioID	Selettore di ID	`p#sotto-titolo`

`E::first-line`
`E::first-letter`
	La prima riga (lettera) formattata dall'elemento E	Pseudo-elemento	`p::first-line {`
			
			`text-transform: capitalize`
			`}`

`E::before`
`E::after`
	Elemento F discendente di un elemento E	Pseudo-elemento	`q::before {`
			`  content: "<<";`
			`}`

`E > F`	Elemento F figlio di un elemento E	Combinatore di possibilità	`tr > th`
`E F`	Elemento F discendente di un elemento E	Combinatore di possibilità	`table th`
`E[foo="bar"]`	Un elemento E con un attributo foo con valore uguale a bar	Attributi	`table[border="1"]`

`E:hover`	Un elemento E ha il puntatore sopra di esso	Pseudo-classe	`a:hover {`
			`  cursor: pointer;`
			`}`

`E:visited`	Un elemento E che è un link già visitato	Link	`a:visited {`
			`  color: gray;`
			`}`

`E:nth-child(n)`	Un elemento E che è l'n-esimo figlio di suo padre	`p:nth-child-(odd)`	`E:nth-child(n)`
`E:first-child`	Un elemento E che è il primo figlio	`h1:first-child`	`E:first-child`

Elenco esaustivo: https://www.w3schools.com/cssref/css_selectors.asp

6.1.3.

6.2. Proprietà di Base

Segue un elenco delle proprietà del CSS maggiormente usate e fondamentali per la costruzione di interfacce web.

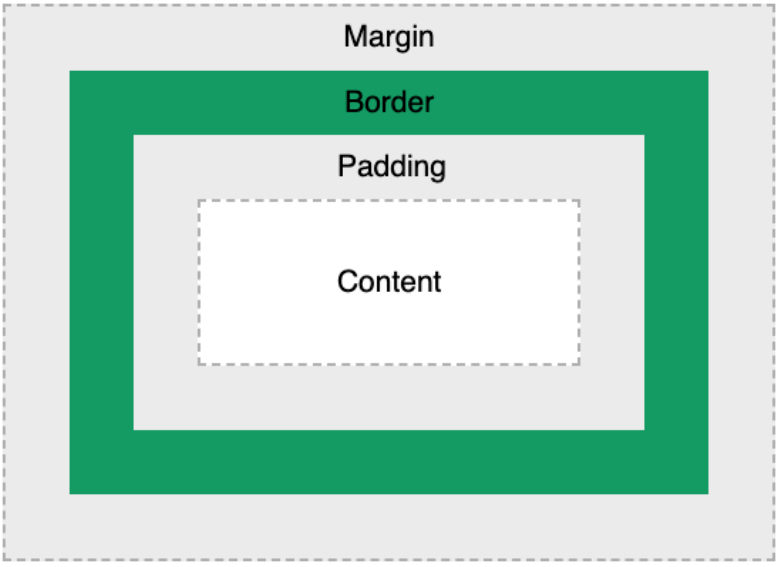
6.2.1. Box

La visualizzazione di un documento con CSS avviene identificando lo spazio di visualizzazione di ciascun elemento.

Ogni elemento è presentato da un scatola rettangolare (appunto il box) che ne contiene il contenuto, quindi i contenitori possono occupare diverse posizioni nello spazio di visualizzazione.

Indipendentemente dalla loro posizione, tutte le scatole CSS sono caratterizzate da quattro elementi:

- **Margine:** la regione che separa un contenitore dall'altro, sempre trasparente (con lo sfondo della scatola contenitore).
- **Bordo:** la regione dove visualizzare un bordo per la scatola.
- **Padding:** la regione di intermezzo tra il bordo della scatola ed il contenuto. Ha sempre il colore di sfondo del contenuto.
- **Contenuto:** la regione dove risiede il contenuto dell'elemento.



Esistono, di conseguenza, le corrispondenti proprietà per controllare questi elementi, inoltre una su una Box in CSS è possibile definire il colore del testo (ereditato dagli elementi che contiene) e background (che invece non viene ereditato). Utilizzando la proprietà `border-radius` è possibile arrotondare i bordi del contenitore.

6.2.2. Forme abbreviate

Il CSS permette di utilizzare delle forme abbreviate per descrivere determinate proprietà, per ottenere questo risultato si utilizza una sequenza separata da spazi di valori, secondo un ordine prestabilito (senso orario per le box). Se si mette un valore solo esso viene assunto da tutte le proprietà individuali.

```
p {
      margin: 1em 2em 3em 4em;
}

p {
      margin-top: 1em;
      margin-right: 2em;
      margin-bottom: 3em;
      margin-left: 4em;
}
```

6.2.3. Layout e posizione delle Box

Le Box sono in relazione alle altre e la loro visualizzazione nello spazio è legata a due aspetti:

- **flusso:** Indica il modo in cui una scatola è posta rispetto alle altre e all'interno della scatola che la contiene.
- **posizione:** Indica la posizione della scatola rispetto al flusso e alle altre scatole.

Il termine Layout indica la disposizione degli oggetti in modo schematizzato nella pagina. Questo concetto ha come compito quello di organizzare gli elementi secondo le loro *dimensioni* e la loro *posizione* in relazione agli altri oggetti presenti.

Se il contenuto richiede più spazio di quello disponibile si applicano tre concetti:

- **resizing:** si cambiano le dimensioni degli oggetti.
- **pagination:** si organizzano gli elementi in più pagine.
- **scrolling:** solo una parte dei contenuti è visibile e si fornisce all'utente un meccanismo per far scorrere i contenuti visibili.

Con il **Layout Responsive** si intende una disposizione ottimizzata per essere letta e visualizzata in maniera coerente su tutti i dispositivi.

6.2.3.1. Viewport

Il canvas è lo spazio (virtualmente infinito) di posizionamento degli elementi via `CSS`, il `viewport` è la parte del canvas attualmente visibile sullo schermo. Questo elemento dipende dalla dimensione dello schermo, i dispositivi di dimensioni ridotte utilizzano un *viewport virtuale* per calcolare le dimensioni degli oggetti nel layout e successivamente le scalano per visualizzare il `viewport reale`.

Per controllare il viewport è stato introdotto un elemento META di HTML, specificando questa proprietà si possono decidere le dimensioni del viewport e forzare il livello di zoom iniziale:

```
<meta name="viewport" content="vidth=device-width",
initial-scale="1.0">
```

6.2.3.2. Unità di misura basate su viewport

La più piccola unità di misura indirizzabile sullo schermo è il pixel (px), le dimensioni dipendono fortemente dalla risoluzione degli schermi e sono molto diversi tra device e device, di conseguenza i pixel non sono una unità di misura affidabile.

Come soluzione a questo problema si possono utilizzare unità di misura che fanno riferimento al viewport:

- **Viewport width** (`vw`): una percentuale (`1%`) della lunghezza attuale del viewport. Ad esempio in un viewport largo `500` pixel, `3vw = 15pc`.
- **Viewport height** (`vh`): una percentuale (`1%`) dell'altezza attuale del viewport.

I **flussi** CSS indicano il modo in cui una box è posta rispetto alle altre. Esistono diversi flussi, tra cui quelli base che sono:

- **Block:** le Box sono poste l'una sull'altra in successione verticale (come paragrafi).
- **Inline:** le scatole sono poste l'una accanto all'altra in successione orizzontale (come parole della stessa riga).
- **Float:** le scatole sono poste all'interno del contenitore e poi spostate all'estrema sinistra o destra della scatola, lasciando ruotare le altre intorno.

Ogni elemento HTML ha un flusso di default che può essere sovrascritto tramite la proprietà `display`.

6.2.3.3. Posizionamento

La posizione dipende dalle altre Box, dallo sfondo (canvas) o dalla finestra (viewport).

Per l'ordinamento delle Box si utilizza la proprietà `position`:

- Posizionamento **`static`** (default): la Box viene posta nella posizione di flusso normale che avrebbe naturalmente.
- Posizionamento **`absolute`**: la Box viene posta nella posizione specificata indipendentemente dal flusso (e nasconde ciò che sta sotto).
    - *La posizione è calcolata rispetto al primo elemento contenitore (padre o ancestor) che ha posizione assoluta o relativa.*
- Posizionamento **`fixed`**: la Box viene posta in una posizione assoluta rispetto alla finestra (viewport), senza scrollare mai.
- Posizionamento **`relative`**: la scatola viene spostata di un delta dalla sua posizione naturale.
    - *Gli elementi figlio possono essere posizionati in modo assoluto. Si una quindi anche senza "spostare" l'elemento ma solo per aver un contenitore per posizionare gli elementi figlio.*
    - *Utile anche per poter usare z-index che non si applica ad elementi static.*
- Posizionamento **`sticky`**: la Box viene posta nella sua posizione naturale all'interno del

suo contenitore, ma durante lo scrolling sta fissa rispetto al viewport fino a che il contenitore non esce dalla vista.

Definito il tipo di posizione, si possono controllare coordinate e dimensioni con altre proprietà:

- **`top, bottom, right, left:`** coordinate della Box (rispetto al contenitore padre).
- **`width, height:`** dimensioni usabili.

Se si indicano le 4 proprietà, la Box verrà creata delle dimensioni indicate, ma il contenuto in eccesso potrebbe fuoriuscire. Per ovviare a questo problema si utilizza la proprietà `overflow`:

- **`visible`**: la Box si espande per contenere tutto il contenuto.
- **`hidden`**: il contenuto extra viene nascosto.
- **`scroll`**: compare una scrollbar per l'accesso al contenuto extra.

6.2.3.4. Visibilità degli elementi

Esistono due chiavi principali per nascondere o rendere visibili gli elementi:

- `visibility:`
  - `visible`
  - `hidden o collapse:` l'elemento viene aggiunto al DOM ma non è visibile.
- `display:`
  - `none:` l'elemento non è aggiunto al DOM e lo spazio che avrebbe occupato può essere occupato da altre Box.

Le Box possono sovrapporsi, ad esempio in caso di `position: relative` o `absolute` (e dropdown o tooltip), per ovviare (o gestire) questa situazione si utilizza la proprietà `z-index`, che permette di indicare la posizione di una Box nella pila di Box sovrapposte. Più è alto il valore più l'elemento è in primo piano.

La proprietà `float` indica che una Box deve essere spostata a sinistra o a destra all'interno del contenitore e le altre Box ruotarle intorno.

6.2.3.5. CSS Flexbox

Il modulo CSS Flexbox è uno strumento estremamente flessibile per costruire layout fluidi senza usare `float`. Questo componente introduce il valore `flexbox` per la proprietà `display`.

Le proprietà di container e item sono usate per decidere:

- Direzione in cui disporre gli item.
- Possibilità di disporre gli item su più linee.
- Allineamento, spaziature e ordine degli item.

Le proprietà di un flex container sono:

- `flex-direction`: definisce l'asse principale e può assumere valori `row`, `column`, `row-reverse`, `column-reverse`.
- `flex-wrap`: elementi su una o più linee, con i rispettivi valori: `wrap`, `nowrap`.

- `justify-content`: allineamento sull'asse principale, valori possibili: `flex-start`, `flex-end`, `center`, `space-between`.
- `align-items`: allineamento sull'asse secondario, con i possibili valori: `start`, `end`, `center`, `stretch`.
- `gap`: spazio tra i `flex items`.

Ogni elemento può avere un valore `order`, per specificare l'ordine di visualizzazione (`order: 1, order: 2, order: 3`).

Esistono altre proprietà che controllano come distribuire lo spazio rimanente dopo aver posizionato gli elementi:

- `flex-grow`: capacità di aumentare le dimensioni lungo l'asse principale.
- `flex-shrink`: capacità di ridurre le dimensioni.
- `flex-basis`: assume la dimensione di default prima di ridistribuire lo spazio.

6.2.3.6. CSS Grid

Il modulo CSS Grid permette di costruire layout basati su una struttura a griglia, formata quindi da righe e colonne su cui sono disposti gli elementi, anche in questo caso si usa la proprietà display: grid e il layout è formato da un contenitore (grid container) e un insieme di grid item. Le proprietà di questi elementi permettono di controllare accuratamente il modo in cui la griglia è costruita e i contenuti sono disposti e visualizzati.

Per definire lo spazio occupato da righe e colonne:

```
- grid-template-rows: 1fr 4fr
- grid-template-columns: 100px 300px 400px
- grid-template-columns: 2fr 2fr 1fr
```

Per posizionare un elemento su una o più righe e/o colonne:

```
- grid-column-start: 2
- grid-column-end: 3
- grid-row-start: 1
- grid-row-end: span 3
```

Oppure usando shorthand (start, end):

```
- grid-column: 2 / 4
- grid-row: 1 / 3
```

Si possono anche definire template che usano i nomi delle aree per indicare su quali righe e/o colonne saranno posizionate, tramite la proprietà `grid-template-areas`.

Le aree saranno poi identificate tramite i valori della proprietà `grid-area` di ogni `grid item`.

6.2.3.7. Proprietà del testo

Il testo, quindi paragrafi o interlinee, può essere modificato tramite le seguenti proprietà (ereditarie se passate tramite un contenitore padre):

- `font-family`: il nome del font, possono essere multipli (fallback).
- `font-style`: `normal`, `italic`, `oblique`.
- `font-variant`: `normal`, `small-caps`.
- `font-weight`: `normal`, `bold`, `bolder`, `lighter`, `100 <-> 900`.
- `font-stretch`: `normal`, `wider`, `narrower`.
- `text-align`: allineamento del testo, `start`, `end`, `center`, `justify`.
- `text-decoration`: `none`, `underline`, `overline`, `line-throught`, `blink`.
- `text-shadow`: ombre per il testo.
- `text-transform`: `capitalize`, `uppercase`, `lowercase`, `none`.
- `letter-spacing / word-spacing`: spaziatura tra lettere e tra parole.
- `white-space`: `normal`, `pre`, `nowrap`, specifica la gestione dei ritorni a capo e del collassamento dei whitespace all'interno di un elemento.

6.2.3.8. Liste

Le liste possono essere formattate tramite CSS con le seguenti proprietà:

- `list-style-image`: immagine da usare come marker.
- `list-style-position`: posizione del marker rispetto al testo (`inside`, `outside`).
- `list-style-type`: tipo di marker, ad esempio `square`, `lower-latin`, `lower-roman`…

6.2.3.9. Tabelle

CSS inoltre mette a disposizione alcune proprietà per formattare e applicare uno stile anche alle tabelle:

- `border`: per definire un bordo, come per le Box.
- `border-collapse`: indica se i bordi devono essere collassati (`separate` / `collapse`).
- `border-spacing`: spazio tra i bordi di due celle adiacenti.
- `caption-side`: posizione della didascalia (`top` / `bottom`).

6.2.4. Responsive Layout, Media Query

Le box che compongono i layout possono essere ridimensionate in maniera automatica tramite l'uso delle percentuali o del relativo viewport.

Questo approccio però non è sempre soddisfacente dal punto di vista grafico, infatti si potrebbe avere la necessità di adattare i propri layout a schermi molto ampi o contrariamente piccoli. Per risolvere questo problema si utilizzano le `Media Query` che permettono di avere una ulteriore flessibilità dal punto di vista dei layout.

Questi componenti servono per specificare delle regole particolari che vengono attivate nel caso in cui il supporto usato per visualizzare la pagina web soddisfa determinati vincoli.

Tramite il loro utilizzo si possono adattare automaticamente tutte le pagina web a tutti i dispositivi indipendentemente dalla loro dimensione senza effettivamente dover **modificare il**

layout della pagina.

La sintassi che compone questi elementi è la seguente:

```
@media <query> {
      selettore {
            attributi
      }
      selettore {
            attributi
      }
}
```

Nel parametro `<query>` è possibile inserire una condizione, che si basa su due principi:

- tipo di dispositivo: `print`, `screen`, `speech`.
- caratteristiche (> 25 valori): `aspect-ratio`, `width`, `height`, `hover`, `monochrome`…

Queste condizioni possono essere combinate con operatori logici: `and`, `or`, `not`.

```
@media print {  }

@media screen and (min-width: 300px) {  }

@media screen and (min-width: 400px) and (max-width: 1024px) {  }
```

Per creare layout responsive si individuano **break-point**, ossia punti di cambiamento del layout, in corrispondenza con larghezze dello schermo diverse. Usando i **Media Query** è possibile specificare il comportamento del layout prima e dopo il breakpoint (con le condizioni di `min-width` e `max-width`).

Gli standard classici sono:

- Portrait mobile: `< 576px`
- Landscape mobile: `>= 576px`
- Portrait Tablets (con navbar collassata): `>= 768px`
- Landscape Tablets: `>= 992px`
- Laptops, Desktops, TVs: `>= 1200px`

7. JavaScript

JavaScript è un linguaggio di scripting client-side, nato nel 1995 ed introdotto da Sun e Netscape.

Ottiene la sua prima standardizzazione (ECMA) nel 1997 e per i seguenti 18 anni sono state applicate solamente piccole estensioni e modifiche.

Javascript ha subito diverse evoluzioni e grazie ad alcune librerie (come NodeJS) ora è possibili eseguire uno script anche lato server:

- **client-side**: tramite eventi, ogni elemento del DOM ha alcuni eventi associati (click,

`mousehover`, `doppio click`, `tasto di tastiera`..), e degli attributi `on+evento` associati.
- **server-side**: tramite routing, i servizi lato server sono associati a URI. Il modo più antico e semplice è creare servizio diverso e inserirli in file separati, ciascuno con un URI proprio. Aprendo poi una connessione su un determinato URI allora viene invocato lo script ed eseguito. NodeJS utilizza un comportamento simile a quello descritto, ad ogni connessione lo script centrale esegue la funzione corrispondente.

Gli script che vengono chiamata lato browser possono essere eseguiti in due modi:

- In maniera **sincrona**: quindi appena lo script viene letto, in un tag `<script>` o in un file dedicato.
- In maniera **asincrona**:
  - Associando il codice ad un evento che viene scatenato tramite il DOM (il click di un determinato bottone): *event-oriented processing.*
  - Associando il codice al completamento di un'operazione di rete (chiamata HTTP / HTTPS). Queste operazioni Ajax vengono gestite tramite *callback,* funzioni eseguite appena la richiesta HTTP asincrona è completata e i dati sono stati ricevuti.
  - Associando il codice ad un timeout, ovvero un periodo di attesa dopo il quale lo script viene eseguito automaticamente.

Durante l'esecuzione dello script **il browser è bloccato** quindi non reagisce all'input dell'utente. Per questo gli script devono essere brevi e veloci, in modo da lasciare all'utente la sensazione di interattività e controllo.

7.1. Come attivare gli script

Il formato HTML prevede l'uso di script in tre modi diversi:

- Posizionato dentro all'attributo di un evento.
- Posizionato nel tag `<script>`.
- Indicato in un file esterno puntato al tag `<script>`.

Inoltre l'HTML visualizza l'output degli script in 4 modi diversi:

- Scrivendo direttamente l'output nella finestra del browser:
  - `document.write("string");`
- Scrivendo sulla console:
  - `console.log("string");`
- Scrivendo in una finestra di Alert:
  - `alert("string");`
- Modificando il DOM del documento visualizzato:
  - `document.getElementById("id").innerHTML = "string";`

7.2. Tipi di dato

JavaScript è un linguaggio minimale e flessibile per quanto riguarda i tipi di dato, infatti ci sono solamente tre tipi di dati atomici:

- `booleani`
- `numeri` (sia interi che floating point)
- `stringhe`

Inoltre vanno considerati come tipi di data anche:

- `null`
- `undefined`

C'è poi un unico tipo di dato strutturato, *object,* di cui fanno parte anche gli *array.*

I dati in javascript sono tipizzati, mentre le variabili no:

```
var pippo;
pippo = "ciao";
pippo = 5;
pippo = [1,2,3];
```

Esistono tre modi per definire le variabili:

- `var pippo = 'ciao'`: definisce una variabile nello scope della funzione o del file in cui si trova.
- `let pippo = 'ciao'`: definisce una variabile nello scope del blocco parentetico o della riga in cui si trova.
- `cost pippo = 'ciao'`: definisce una variabile non ulteriormente modificabile.

7.3. Operatori

Come in ogni linguaggio esistono degli operatori matematici e logici predefiniti, che si distinguono in base al tipo di dato:

Numeri			
Operatore	**Descrizione**	**Esempio**	**Commento**
`+`	Somma	`let a = 5 + 7`	a vale 12
`-`	Sottrazione	`let b = 12 - 2`	b vale 10
`*`	Moltiplicazione	`le c = 3 * 2`	c vale 6
`%`	Modulo	`let e = 15 % 6`	d vale 3 (2 resto 3)
`**`	Esponente	`let f = 3 ** 2`	f vale 9
`++`	Incremento	`e++`	e vale (e+1)
`-`	Decremento	`e-`	e vale (e-1)

Stringhe			
+	Composizione	`g = "Hel" + "lo"`	g vale "Hello"
+	Composizione + casting	`h = "5" + 7`	h vale "57"
Booleani			
==	Uguaglianza	`let i = b == c` `let j = 5 == '5'`	i è falso (b = 1, c = 2) j è vero (con casting)
<	Minore	`let k = b < c`	k è vero
>	Maggiore	`let j = b > c`	j è falso
<=	Minore Uguale	`let i = b <= 5`	i è vero
>=	Maggiore Uguale	`let i = b >= 2`	i è falso
!=	Disuguaglianza	`let o = b != c`	o è vero
===	Uguaglianza **senza casting**	`let q = 5 === '5'`	q è falso
!==	Disuguaglianza **senza casting**	`let r = 2 !== '2'`	q è vero
&&	And	`let s = i && j`	s è falso
||	Or	`let s = k || i`	s è vero
!	Not	`let u = !j`	u è vero

7.4. Strutture di controllo

Esistono, come per gli altri linguaggi, le classiche strutture di controllo. Infatti troviamo i blocchi:

Blocco IF - IF ELSE

```
if (a == 5) {
      do_something
}

if (a < 3) istruzione_singola
```

```
if (4 > 6) {
      sono_qui
} else {
      altrimenti_sono_qui
}
```

Operatore Ternario

```
let x = (b === 5 ? 'allora faccio' : 'altrimenti faccio');
```

Blocco Switch

```
switch (a) {
      case 1: //myIstructionFor1
            break;
      case 2: //myInstructionFor2
            break:
      case 3: //myInstructionFor3
            break:
      default: //myDefaultCase
            break;
}
```

Blocco for…in

```
for (let i = 0; i < k; i++) {
      instruzione_1;
      instruzione_2;
      alert(i);
}
```

Blocco while

```
while (k < 5) {
      altre_cose;
}
```

Blocco do…while

```
do {
      altre_cose;
} while (k < 5)
```

Blocco try…catch

```
try {
      let el = document.getElementById("menu"+x);
      let address = el.attributes["href"].value;
      return address ;
} catch (error) {
      return "input Value out of bounds" ;
}
```

7.5. Funzioni

Le funzioni in Javascript sono blocchi di istruzioni dotati di un nome e facoltativamente di parametri, possono ma non sono obbligate a restituire un valore di ritorno. Le funzioni non sono tipate, i valori di ritorno si.

```
function double(n) {
      let m = n + n;
      return m;
}

let a = double(4);     // a = 8
let b = double('test') // b = 'testtest'
```

Se manca un parametro, non restituisce un errore ma assume che il parametro sia `undefined`.

```
function double(n) {
      if (typeof n != undefined) {
            let m = n + n;
      } else {
            let m = 0;
      }
      return m;
}

let c = double(); // c = 0;
```

In JavaScript le funzioni sono oggetti quasi come tutti gli altri:

- Possono essere assegnate a variabili.
- Possono essere passate come parametri di funzione.
- Possono essere restituite da una funzione.
- Possono essere elementi di un *object*.
- Possono essere elementi di un *array*.

Inoltre possono essere invocate con l'operatore `()`.

Assegnazione variabile:

```
let potenza = function(a,b) {
      return Math.pow(a,b);
}
let c = potenza(5,3);
```

Assegnazione di una funzione come metodo di un oggetto:

```
var e = { p:3, q:5, r:7 }
e.sum = function() {
      return this.p + this.q + this.r;
}
```

Il nome semplice è una variabile, se si aggiungono le parentesi diventa un invocazione e la funzione viene eseguita:

```
if (e.sum) { let c = e.sum(); }
```

Inoltre si può assegnare una funzione come se fosse una proprietà di un oggetto o di un array e richiamarla in modo naturale:

```
let persona = {
      nome: 'Giovanni',
      cognome: 'Verdi',
      saluta: function(name, id) {
            let saluto = "Ciao " + name;
            if (id) {
                  document.getElementById(id).innerHTML = saluto;
            }
      }
}
```

7.5.1. Arrow Function

Le funzioni freccia (arrow function) sono composte da una sintassi leggermente differente, sono utili per definire funzioni semplici di callback, ad esempio nelle funzioni filtro degli array o negli eventi:

```
let arr = [1, 2, 3];
let squares = arr.map(x => x * x);

let showHelp = document.getElementById('showHelp');
let helpDiv = document.getElementById('helpDiv');
showHelp.onclick = () => helpDiv.classList.toggle('d-none');
```

7.6. Tipi di dati strutturati

Javascript ha un unico tipo di dato strutturato, chiamato *object*. Anche gli *array* sono un tipo di dato speciale *object*. Questi elementi sono liste non ordinate di proprietà coppie nome-valore.

```
let persona = {
      nome: 'Paolo',
      cognome: 'Rossi',
      altezza: 185,
      nascita: new Date(1998, 3, 13)
}
```

Il valore di una proprietà può essere esso stesso un *object*:

```
let persona = {
      …attributi normali,
      indirizzo: {
            via: {
                  strada: 'Via Roma',
                  numeroCivico: 12
            },
            citta: 'Bologna',
            nazione: 'Italia'
```

```
    }
}
```

Per accedere alle proprietà di un *object* Javascript mette a disposizione due metodi principali che prendono il nome:

- *Dot Syntax*: con ispirazione ai linguaggi orientati ad oggetti.
  - `alert(persona.nome + ' ' + persona.cognome);`
- *Square Bracket Syntax*: che prende ispirazione dagli array associativi.
  - `alert(persona['nome'] + persona['cognome']);`
  - In questo caso il nome della proprietà è una normale stringa:
    - ```
      let n1 = 'nome';
      alert(persona[n1]);
      ```

Per accedere ai livelli sottostanti di un oggetto basta aggiungere `.` oppure `[nome]` dopo ogni estrazione.

```
persona.indirizzo.via.numero = '36' ;
persona['indirizzo']['via']['numero'] = '15/a' ;
```

Queste sintassi si possono usare sia per leggere sia per scrivere le proprietà di un oggetto.

7.6.1. Array

Un array è un oggetto in cui le chiavi sono numeri interi assegnati automaticamente. Per distinguerlo da un oggetto normale si usa la parentesi quadra invece che la parentesi graffa.

```
let nomi = ['Andrea', 'Beatrice', 'Carlo'];
```

La *Dot Syntax* non può essere usata, ma solo quella bracket. Il primo numero è lo zero.

```
nomi[0] = 'Adriano';
```

Un array essendo un oggetto contiene alcuni metodi molto utili:

- `length`: lunghezza dell'array.
- `indexOf(item)`: la posizione di *item* nell'array.
- `pop()`: toglie un valore in fondo all'array e lo restituisce.
- `push(item)`: aggiunge *item* in fondo all'array.
- `shift()`: toglie un valore in cima all'array e lo restituisce.
- `unshift()`: aggiunge *item* in cima all'array e restituisce la nuova lunghezza.
- `slice(start, end)`: restituisce un array da start a end (escluso):
  - `let b = nomi.slice(1, 2); // b = ['Beatrice']`
- `splice(pos, rimuovi, inserisci)`: inserisce e rimuove elementi:
  - ```
    let pers = ["Andrea", "Barbara", "Carlo", "Elena"];
    pers.splice(2, 1, "Claudio");
    persone == ["Andrea", "Barbara", "Claudio", "Elena"];
    ```
- `join(sep)`: crea una stringa usando *sep* come separatore:
  - ```
    let p = pers.join(", ");
    p == "Andrea, Barbara, Claudio, Elena";
    ```
    ```

Altre funzioni *filtro* che possono essere utilizzate con gli array:

- `array.sort(f)`: restituisce un array ordinato sulla base della funzione f, che deve avere due parametri e restituire un valore 1 (stesso ordine), -1 (inverti ordine).
- `array.filter(f)`: restituisce un secondo array che contiene solo gli elementi che soddisfano la funzione `booleana` f.
- `array.some(f) - array.every(f)`: restituisce vero se almeno un elemento (`some()` o tutti gli elementi `every()`) soddisfano la funzione `booleana` f.
- `array.find(f)`: restituisce il primo elemento che soddisfa la funzione booleana f.
- `array.forEach(f)`: esegue sull'array la funzione f permettendo di modificare l'array direttamente.
- `array.map(f)`: crea un nuovo array in cui ogni elemento viene modificato dalla funzione f.
- `array.reduce(f)`: esegue su ogni elemento dell'array la funzione f passando il risultato dell'esecuzione precedente. Ottimo per calcoli.

7.6.2. Oggetti predefiniti

Javascript predefinisce alcuni oggetti utili per raccogliere insieme i metodi più appropriati per certi tipi di dati, alcuni oggetti multipli sono:

- Object
- Array
- String
- Date
- Number
- RegExp

Inoltre esistono anche altri tipi di oggetti come:

- Math
- JSON

7.6.2.1. Stringhe

L'oggetto *String* contiene metodi disponibili per tutti i valori di tipo stringa:

```
let str = 'Precipitevolissimevolmente';
```

- `length`: lunghezza della stringa.
  - `let s = str.length; // s = 26`
- `indexOf(sub)`: la posizione di *sub* nella stringa.
  - `let t = str.indexOf('ss') // t = 13`
- `substring(start, end)`: restituisce la sottostringa da *start* ad *end*.
  - `let x = str.substring(3,8); // x = 'cipit'`
- `substr(start, length)`: restituisce la sottostringa da *start* per *length* caratteri.
  - `let y = str.substr(3,8); // y = "cipitevo"`
- `split(sep)`: separa una stringa in array utilizzando *sep* come separatore.
  - `let w = "130.136.1.110" ;`

```
let z = w.split(".") ; // z = ['130', '136', '1', '110']
```

7.6.2.2. JSON

JSON (Javascript Object Notation) è un formato dati derivato dalla notazione usata da JS per gli oggetti:

```
{
      “nome”: ["Giuseppe" , "Andrea" , "Federico"],
      “cognome”: “Rossi”,
      “altezza”: 180,
      “indirizzo”: {
            “via”: {
                  “strada”: “Via Indipendenza”,
                  “numero”: 15
            }
      }
      …
}
```

Rispetto alla notazione utilizzata nei programmi bisogna sottolineare che:

- Esistono solo valori string, number, boolean, array, object.
- Anche i nomi delle proprietà sono tra virgolette.
- Si usano solo le virgolette doppie e non le semplici.
- Non si possono inserire commenti di nessun tipo.

Tutti i linguaggi di programmazione più importanti oggi accettano e scrivono dati in formato JSON.

7.6.2.3. Date

Una data è un oggetto che esprime un giorno e un orario rappresentandolo come il numero di millisecondi trascorsi dalla mezzanotte del 1 Gennaio 1970 e la data in questione. Poiché in realtà è un numero, questo permette di fare operazioni aritmetiche e confronti numerici:

- Costruttore:
  - `let d = new Date(); // Contiene la data e l’ora attuale`
    `let capodanno = new Date(2025, 0, 1); // I mesi partono da 0`
- `getDay()`: ritorna il giorno della settimana (0 Domenica - 6 Sabato).
- `toDateString()`: converte il numero in una stringa leggibile:
  - `let a = new Date().toLocateString(); // “10/06/2025”`
    `let a = new Date().toDateString(); // “Thu Jun 10 2025”`

7.6.2.4. Altri oggetti

Altri oggetti possono appartenere al costrutto Math ad esempio:

- `Math.PI()`, `Math.abs()`, `Math.sin()`, `Math.cos()`, `Mat.round()`, `Mat.sqrt()`, `Math.random()...`

Un altro costrutto molto potente è RegExp che serve per definire espressioni regolari, utilizzabili per fare match e controlli su stringhe (usano / come delimitatore):

```
let str = 'Precipitevolissimevolmente';
let re = /pi(.)/ ;
let x = str.match(re) ;
let y = re.exec(str) ;
```

7.7. Javascript & Browser

L'oggetto *window* è l'oggetto top-level con le proprietà e i metodi della finestra principale:

- Posizione: `moveBy(x, y), moveTo(x, y)...`
- Dimensioni: `resizeBy(x, y), resizeTo(x, y)...`
- Altre finestre: `open("URLName", "WindowName",["opt"])...`

Un altro oggetto che influisce sulle proprietà del client è *navigator*, che contiene nome, numero di versione, plug-in installati, supporto per i cookies.

Un altro elemento fondamentale è *location* che contiene l'URL del documento corrente. Modificando questa proprietà il client accede a un nuovo URL (redirect):

- `window.location = "https://ilmiosito.com/"`
- `window.localtion.href = "https://ilmiosito.com/"`

Un altro oggetto fondamentale è il *document* che rappresenta il contenuto del documento, ed ha proprietà e metodi per accedere ad ogni elemento della gerarchia:

- `document.title`: titolo del documento.
- `document.forms[0]`: il primo form.
- `document.forms[0].checkbox[0]`: la prima checkbox del primo form.
- `document.forms[0].check1`: l'oggetto con nome *check1* nel primo form.
- `document.myform`: l'oggetto *myform*.
- `document.images[0]`: la prima immagine.

Inoltre *document* rappresenta l'oggetto `DOMDocument` del DOM del documento visualizzato.

Ogni oggetto di questa gerarchia è caratterizzato da un insieme di proprietà, metodi ed eventi che permettono di accedervi, controllarlo e modificarlo. Javascript nacque per controllare i valori di un form:

```
function verify() {
      if (document.forms[0].elements[0].value == "") {
            alert("Il nome è obbligatorio!");
            document.forms[0].elements[0].focus();

            return false;
      }
}

<form action="..." onSubmit="verify()">
```

7.7.1. Document Object Model (DOM)

Il Document Object Model è un'interfaccia di programmazione (API) per documenti sia HTML che XML, questo definisce la struttura logica dei documenti ed il modo in cui si accede e si manipola un documento.

Ogni componente di un documento HTML o XML può essere *letto, modificato, cancellato o aggiunto* utilizzando il Document Object Model.

Il core del DOM definisce alcune classi fondamentali per i documenti HTML e XML e ne specifica proprietà e metodi. La classe principale di `DOM` è `DOMNode`, di cui la maggior parte delle altre classi è una sottoclasse.

Le classi principali definite nel DOM sono:

- `DOMDocument`: il documenti di cui si sta parlando.
- `DOMElement`: ogni singolo elemento del documento.
- `DOMAttr`: ogni singolo attributo del documento.
- `DOMText`: ogni singolo nodo di testo del documento.
- `DOMComment, DOMProcessingInstruction, DOMCDATASection, DOMDocumentType`…

7.7.1.1. DOM Node

DOMNode specifica i metodi per accedere a tutti gli elementi di un nodo di un documento, inclusi il nodo radice, il nodo documento, i nodi elemento, i nodi attributo, i nodi testo…

I metodi principali sono:

- `insertBefore()`
- `replaceChild()`
- `removeChild()`
- `appendChild()`
- `hasChildNodes()`
- `hasAttributes()`

7.7.1.2. DOM Document

DOMDocument specifica i metodi per accedere al documento principale, è l'equivalente della radice dell'albero (non elemento radice). Contiene i metodi:

- `createElement()`
- `createAttribute()`
- `createTextNode()`
- `getElementsByTagName()`
- `getElementById()`

7.7.1.3. DOM Element

DOMElement specifica i metodi e i membri per accedere a qualunque elemento del documento. Semplificando:

- getAttribute()
- setAttribute()
- removeAttribute()

7.7.1.4. Javascript e DOM

Javascript implementa i metodi standard per accedere al DOM del documento:

```
var c = document.getElementById('c35');

c.setAttribute('class', 'prova1');
c.removeAttribute('align');

var newP = document.createElement('p');
var text = document.createTextNode('Hello World');

newP.appendChild(text);
c.appendChild(newP);
```

Esempio creazione di una lista:

```
// Creazione elementi singoli
Olist = document.createElement('ol');
voce1 = document.createElement('li');
voce2 = document.createElement('li');
testo1 = document.createTextNode('Just a simple text');
testo2 = document.createTextNode('Other text');

// Creazione lista completa
voce1.appendChild(testo1);
voce2.appendChild(testo2);
Olist.appendChild(voce1);
Olist.appendChild(voce2);

// Inserimenti ista in una data posizione
div = document.getElementById('lista');
body = document.getElementByTagName('body').item(0);
body.insertBefore(Olist, div);
```

7.7.1.5. innerHTML & outerHTML

Il `DOM` per `HTML` permette di leggere / scrivere interi elementi, trattandoli come stringhe:

- `innerHTML`: legge / scrive il contenuto di un sottoalbero (escluso il tab dell'elemento radice).
- `outerHTML`: legge / scrive il contenuto di un elemento (incluso il tag dell'elemento radice).

Esempio:

```
<div id="para1">
     <p>Paragrafo!</p>
</div>
```

```
let a = document.getElementById(“para1”);
let b = a.innerHTML;
let c = a.outerHTML;
```

La variabile b vale:

```
“<p>Paragrafo!</p>”
```

La variabile c vale:

```
<div id="d"><p>Paragrafo!</p></div> "
```

Modifica dei valori attuali:

```
a.innerHTML = "<ul><li>Lista!</li></ul>";
```

7.7.1.6. Navigare sul DOM

Il DOM non ha metodi propri per navigare sull’albero liberamente. L’unico modo *ufficiale* è quello di esplorare l’albero attraverso la proprietà `children`.

```
// Trova la n-esima cella della m-esima riga della tabella con ID
dato
function getCell(tId, row, cell) {
      let items = document.body.children;
      let i = 0;

      do while (items[i].id !== tId) {
            i++;
      }

      let j = 0;

      do while (items[i].children[j].nodeName !== ‘TBODY’) {
            j++;
      }

      let rows = items[i].children[j].children;

      return rows[row].children[cell];
}
```

7.7.2. Selettori HTML in DOM

I metodi base in DOM per accedere ai nodi di un documento sono:

- `getElementById`: solo se l’elemento ha un id:
  - `let a = document.getElementById(‘myId’);`
- `getElementsByName`: se l’elemento ha un attributo name:
  - `let a = document.getElementsByName(‘nameField’);`
- `getElementsByTagName`: tutti gli elementi con un nome specificato:
  - `let a = document.getElementsByTagName(‘input’);`

Le funzioni `getElementBy[element]` restituiscono `NodeList`, non array, per risolvere questo problema si utilizza `Array.from`:

```
let d = Array.from(document.getElementByClassName('input'));
```

Esistono altri selettori creati dopo il grande successo di JQuery:

- `getElementByClassName`: cerca tutti gli elementi in classe specificata:
  - `let a = document.getElementByClassName('info');`
- `querySelector`: accetta un qualunque selettore CSS e restituisce il primo elemento trovato:
  - ```
    let a = document.querySelector("#para1");
    let b = document.querySelector("[name='NameField']");
    let c = document.querySelector("input");
    let e = document.querySelector(".info");
    ```
- `querySelectorAll`: accetta un qualunque selettore CSS e restituisce tutti gli elementi trovati con una `NodeList`:
  - `let d = Array.from(document.querySelectorAll("input"));`

7.8. Object Orientedness in JavaScript

Javascript è un linguaggio object-oriented anche se non è tipizzato come Java, infatti gli oggetti non sono basati su un concetto di classe ma di **prototipo**.

Inoltre le funzioni sono oggetti di primo livello, di conseguenza ogni oggetto può contenere al suo interno delle funzioni, senza ricorrere alla classe.

Le classi quindi non sono entità di primo livello, al loro posto si utilizzano degli oggetti che provengono dallo stesso costruttore. Si utilizza la keyword *new* per costruire un nuovo oggetto. Inoltre *this* è la keyword utilizzata per riferirsi sempre ad un oggetto.

All'interno di una classe this si riferisce sempre all'istanza:

```
// Costruttore
function Persona(n) {
     this.nome = n;
}
```

All'interno di una callback di un evento, this si riferisce sempre all'oggetto che ha ricevuto l'evento:

```
myButton.onclick = function(e) {
     this.innerHTML = "clicked";
}
```

Se faccio bind() esplicito, this è l'oggetto a cui ho fatto bind():

```
function f() { return this.a; }

let g = f.bind({ a: "ciao" });
console.log(g());
```
```

altrimenti *this* è il global *this*, ovvero `window`.

Ogni oggetto in javascript è autonomo e si possono aggiungere tutti i metodi / proprietà che si vuole senza dover modificare la classe.

Per raggiungere questo obiettivo si utilizza l'oggetto *prototype* che serve per usare librerie di oggetti e metodi:

- Estendere le proprietà di un oggetto built-in nel linguaggio
- Estendere le proprietà di oggetti creati in precedenza.

Ogni oggetto javascript ha una proprietà `.prototype` a cui si può aggiungere un membro ed associare una funzione. La modifica del prototipo può avvenire in qualunque momento nell'esecuzione del programma.

Modificare il prototipo cambia non solo le istanze successive ma anche quelle già generate in precedenza:

```
function Persona(nome, altezza) { // Costruttore
      this.nome = nome;
      this.altezza = altezza;
      this.nascita = nascita;
}

let mario = new Persona("Mario", 178);

Persona.prototype.welcome = function() {
      alert("Benvenuto, " + this.name + "!");
}

mario.welcome(); // Che esiste per le istanze già create
```

7.9. Scope delle variabili, closure e IIFE

Javascript ha tre modi per definire le variabili e quattro tipi di scope:

Variabili globali: ogni variabile definita esternamente alle funzioni, indipendentemente dall'operatore utilizzato:

```
var a = 5;
var b = 4;

function foobar() {
      return a + b;
}
```

Solamente in ambiente Browser le variabili globali sono considerate membri dell'oggetto predefinito window.

```
var a = 5;
console.log(a === window.a);
```

Variabili di modulo: ogni variabile definita esternamente alle funzioni ma in un file etichettato come modulo è globale alle funzioni del modulo, ma è locale al modulo:

```
// Saved as file "imported.js"
var a = "Io sono a";

export f: function() {
      return a;
}

// Altro file
import { f } from ('./imported.js');

function g() {
      var b = f();
      console.log(b); // io sono a
      console.log(a); // undefined
}
```

Variabili locali: ogni variabile definita con var internamente ad una funzione è locale alla funzione, lo scope locale sovrascrive lo scope globale:

```
var a = 5;
var b = 7;

c = 9;

function foobar() {
      var a = 10;
      var d = 13;

      return a + d;
}

console.log(foobar() - c) // 4
```

Variabili di blocco: ogni variabile definita con let o const internamente ad un blocco parentetico è locale al blocco ed è definita da quel momento in poi, lo scope di blocco sovrascrive gli altri scope:

```
var a = 5;

function foobar(b) {
      if (b > 3) {
            let a = 10;
            return a + b;
      }

      return a + b;
}

console.log(foobar(2));
console.log(foobar(5));
```

Javascript inoltre non ha protezione dei membri privati di un oggetto, ma sono tutti accessibili e manipolabili. Ad esempio, se definiamo una classe `Counter` con uno stato privato accessibile attraverso l'interfaccia data dalle funzioni `incrementa()` e `decrementa()`:

```
Counter = function() {
      this.state = 0; // Variabile potenzialmente privata

      this.incrementa = function() {
            return ++this.state;
      };

      this.decrementa = function() {
            return -this.state;
      };
}

var c = new Counter();
c.incrementa();
var m = c.incrementa(); // M in questo momento vale 2
c.state = 7;
var n = c.incrementa(); // N in questo momento vale 8
```

Quindi non è possibile proteggere dati o funzioni all'interno di una Classe.

Oltre ai quattro scope visti in precedenza è presente un ulteriore scope detto *closure*, che rappresenta una funzione all'interno del quale viene definita un'altra funzione.

Ad esempio, una funzione che restituisce una funzione ha uno *scope* che è sempre accessibile alla funzione interna, ma non al mondo esterno.

In questo modo riesco ad ottenere delle variabili interne effettivamente private:

```
Counter = function() {
      var state = 0; // Privata

      return {
            incrementa: function() { return ++state; },
            decrementa: function() { return -state; }
      }
}

var c = new Counter();

c.incrementa();
var m = c.incrementa(); // M vale 2

c.state = 7; // Azione lecita ma INUTILE
var n = c.incrementa(); // N vale 3
```

Inoltre una funzione IIFE (Immediately Invoked Function Expression) è una funzione anonima creata ed immediatamente invocata. Serve sostanzialmente per fare *singleton* (oggetti non ripetibili) dotati di closure (e quindi di stato interno privato). L'oggetto JQuery è il risultato di un IIFE, e così moltissime librerie utilizzano IIFE.

Esempio:

```
var people = (function() {
      var p = [];

      return {
            add: function(per) { p.push(per); },
            list: function() { return p.join(' '); }
      };
}) ()
```

L'oggetto `people`, in questo caso, è un singleton con un array come stato interno e due metodi per accedere e modificare i valori. Per merito della *closure*, la variabile `p` è accessibile da `add` e `list` ma non dall'esterno.

La coppia di parentesi alla fine invoca la funzione immediatamente e senza side effect.

8. AJAX

AJAX (Asynchronous Javascript And XML) è una tecnica per creazione di applicazioni Web interattive, infatti permette l'aggiornamento asincrono di porzioni di pagine HTML garantendo:

- Interattività
- Velocità
- Usabilità

Un'applicazione AJAX è divisa in alcuni momenti chiave:

- Creazione e configurazione delle richieste per il server usando XMLHTTPRequest
- Attivazione della richiesta HTTP
- …attesa
- …ricezione della risposta HTTP e analisi dei dati (o eventuale errore)
- Modifiche al DOM della pagina

Controllo creazione XMLHTTPRequest:

```
if (window.XMLHttpRequest) {
      http_request = new XMLHttpRequest();
}
```

La funzione open invece prepara la connessione HTTP. Esistono due tipi di connessioni, usa sincrona e bloccante oppure asincrona:

```
http_request.open('GET', 'https://mysite.com/file.json', false);
```

Nel caso di una funzione asincrona, prima di attivare la richiesta è necessario specificare la funzione che si occuperà di gestire la risposta e aprire la connessione al server:

```
http_request.onreadystatechange = myHandler;
http_request.open('GET','http://www.example.org/file.json',true);
```

I parametri presenti in 'open' specificano:

- *Il metodo HTTP della richiesta.*
- *l'URL a cui inviare la richiesta.*
- *Un booleano che indica se la richiesta è asincrona.*

La richiesta viene inviata per mezzo di una *send* che contiene il body da inviare come risorsa al server:

```
http_request.send(null);
```

Questo parametro varia in base al tipo di richiesta:

- `POST`: prende la forma di una query string:
  - `name=value&anothername=othervalue&so=on`
- `GET`: ha valore `null`, in questo caso i parametri sono passati tramite l'URL indicato della precedente `open`.

Inoltre può anche essere un qualsiasi altro tipo di dati, in questo caso è necessario specificare il tipo MIME dei dati inviati:

```
http_request.setRequestHeader('Content-Type', 'mime/type');
```

La funzione asincrona incaricata di gestire la risposta deve controllare lo stato della richiesta, che può assumere i seguenti valori (`readyState`):

- `0` = uninitialize
- `1` = loading
- `2` = loaded
- `3` = interactive
- `4` = complete

```
function myHandler() {
      if (http_request.readyState == 4) {
            // Risposta ricevuta
      } else {
            // Risposta non ancora ricevuta
      }
}
```

Una volta ricevuta la risposta è poi necessario controllare lo `status code`:

```
if (http_request.status === 200) {
      // OK
} else {
      // ERROR
}
```

Infine è possibile leggere la risposta inviata dal server utilizzando:

- `http_request.responseText`: che restituisce la risposta come testo semplice.
- `http_request.responseXML`: che restituisce la risposta come XMLDocument.

Esempio sincrono di una chiamata XHR:

```
function getData() {
      try {
            // Attivazione connessione AJAX
            myXHR = new XMLHttpRequest();
            myXHR.open("GET", "http://server.it/api/1", false);
            myXHR.send(null);

            // IN QUESTO MOMENTO IL BROWSER NON ACCETTA INTERAZIONI
            // DA PARTE DELL'UTENTE

            let d = JSON.parse(myXHR.responseText);

            let data = prepareHTML(d); // Funzione esempio
            document.querySelector('#result').innerHTML data;
      } catch (error) {
            alert('Errore');
      }
}
```

Esempio asincrono completo di una chiamata XHR:

```
function getData() {
      // Attivazione connessione AJAX
      myXHR = new XMLHttpRequest();
      myXHR.onreadystatechange = myHandler;
      myXHR.open("GET", "http://server.it/api/1", true);
      myXHR.send(null);

      // BROWSER LIBERO DI ACCETTARE INTERAZIONI DALL'UTENTE
}

function myHandler() {
      if (myXHR.readyState == 4) {
            if (myXHR.status == 200) {
                  let d = JSON.parse(myXHR,responseText);

                  let data = prepareHTML(d); // Funzione esempio
                  document.querySelector('#result').innerHTML data;
            }
      }
}
```

Esempio con implementazioni JQUERY

```
$.ajax({
      method: "GET",
      url: "http://www.server.it/server" ,
      success: function(d) {
            let data = prepareHTML(d);
            $('#result').html(data);
            alert("Caricamento effettuato");
```

```
    },

    error: function(data) {
        alert("Caricamento impossibile");
    }

});
```

Un altro componente fondamentale, introdotto dopo gli standard XML è la funzione `fetch`:

- Una libreria standard dei browser ispirata a JQuery.
- Un altro modo di fare richieste Ajax basato su promesse.
- Ogni singolo passaggio è una promessa, incluso la ricezione e la decodifica della risposta.

```
fetch("http://server.it/api/products")
    .then(response => response.json())
    .then(d => {
        let data = prepareHTML(d);
        document.querySelector('#result').innerHTML = data;
        alert('Caricamento completato');
    }).catch((error) => {
        alert('Errore');
    });
```

Un parametro opzionale permette di configurare la richiesta:

```
fetch("http://server.it/api/products", {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.strigify(formData)
    })
    .then(response => response.json())
    .then(d => {
        let data = prepareHTML(d);
        document.querySelector('#result').innerHTML = data;
        alert('Caricamento completato');
    }).catch((error) => {
        alert('Errore');
    });
```